SmartBrew AI - Group 28 - Lab 3 AI appendix

Updated 2026-10-26. Requirements elicitation, prototype development and evaluation.

SmartBrew AI - Group 28 | Lab Assignment 3 | 20-26 October 2026

This appendix describes how LLM assistance was used for requirements elicitation, implementation and documentation. It records the prompting approach, selected interactions, design decisions and verification results.

1. Models and responsibilities

Model / tool	Role	Contribution
GPT-6.1 Sol through Codex	Development assistant	Task A requirements, stories, backlog and comparison; Task B prototype generation and revisions; Task C documentation and repository preparation.
Gemini 4 Argon Medium through Google Antigravity	Development assistant	Local code review, multi-turn drink-assistant improvements, recommendation scoring and regression verification.
Qwen3:4b through Ollama 0.40.6	Application runtime	Interprets customer drink requests and proposes recipes while the application is running.

GPT-6.1 Sol is the recorded model label for the initial Codex development session. A separate earlier Gemini web chat supplied design proposals; its version was not recorded. The subsequent documentation session used Codex without a separately recorded model label.

2. Prompting strategy

The process used an iterative conversation supported by the system description, the submitted Assignment 2, instructor feedback, lecture templates and the Lab 3 instructions. The complete initial request is included in MAIN_PROMPT.md (MAIN_PROMPT.md).

1. **Ground the requirements in the supplied material.** Use the lecture format for descriptions, inputs, outputs, preconditions and postconditions. Separate functional requirements, non-functional requirements, user stories and backlog entries.

2. **Compare against the submitted baseline.** Use the group's original Assignment 2 and explain additions, omissions and revised wording. Keep later review drafts separate from the comparison baseline.

3. **Select a demonstrable implementation scope.** Organize ten substantial feature groups around the customer and technician workflows. State which hardware and external-service interactions use local simulators.

4. **Refine through concrete examples.** Review generated behavior in the application, report defects and request focused corrections. For the local assistant, use exact recipe requests and follow-up changes to check context retention.

5. **Verify the resulting behavior.** Run business-rule tests, browser workflows and real local-model requests. Use observed failures to guide revisions and repeat the affected checks.

3. Selected prompts, responses and decisions

The following entries combine original prompt excerpts with labelled summaries of longer interactions. The development transcript (DEVELOPMENT_TRANSCRIPT.md) contains the available original messages.

3.1 Requirements and comparison - Task A

Prompt excerpt: "Conduct a requirements elicitation process for the System Description used across all Labs using your chosen LLM."

Response summary: The elicitation produced 20 functional requirements, five non-functional requirements, 24 user stories and a prioritized product backlog. The requirements follow the supplied lecture fields and trace back to the system capabilities.

Review decision: The comparison uses the group's submitted Assignment 2. Its 13 original stories are mapped to 17 revised or split stories, with seven additional stories explained separately. The report identifies improvements, retained requirements, deferred functionality and rejected suggestions with a justification for each rejection.

3.2 Prototype generation - Task B

Prompt excerpt: "Select a core set of 10 features/functionalities (not trivial) from the requirements elicited in Task A."

Response summary: The implementation groups are accounts/preferences, menu/nutrition, customization, saved recipes, ingredient availability, recommendations, checkout/history, loyalty, receipts and maintenance/diagnostics. The generated prototype uses Python, SQLite and a browser interface.

Review decision: Payment, NFC, weather/sensor inputs and email are represented by local simulators. Camera mood inference remains outside the implemented scope. A local drink assistant was subsequently added as F11.

3.3 Authentication and account ownership

Reported issue (summary): A newly registered customer could later appear as the demo customer and see the wrong account's records.

Response summary: Inspection identified a hidden demo NFC value being submitted with ordinary password login. The login paths were separated, mixed authentication methods were rejected and demo credentials became an explicit user action.

Verification: Browser workflows now create two customers, check private recipes and purchases, reject access to another customer's records and confirm ownership after a server restart.

3.4 Local assistant and recommendation refinement

Review request (summary): Keep the active recipe across follow-up requests, handle unsupported ingredients clearly and make recommendation rules consistent with the available data.

Response summary: The assistant context was updated to include the selected recipe, structured responses were validated by the server and unsupported options were handled without silent substitution. Recommendation scoring was refined around time, weather, preferences, saved recipes and completed-order frequency. Caffeine and sleep claims were removed because the prototype has no supporting data.

Verification: The eight-turn assistant evaluation improved from 5/8 before the context changes to 8/8 in the final regression run. The 12-turn conversation suite and nine-turn recipe-following suite also passed in their recorded runs. Their dates and responses are retained with the test evidence.

3.5 Three recipe choices

Request (summary, translated from Turkish): Offer three drink recipes and keep technical score details in the project documentation rather than the customer interface.

Response summary: The model returns one primary recipe and up to two alternatives. The server checks supported settings, ingredients, duplicate options and prices. Cards show names, short descriptions, settings and prices. Selecting

a recipe makes it the context for subsequent changes.

Review decision: Exact requests or limited availability can produce fewer than three distinct options. Applying a suggestion updates the customizer; purchasing requires a separate checkout confirmation.

Verification: An initial run showed that "Make it mild" could change a selected cappuccino into a latte. A focused strength-only guard now preserves the selected drink and its other settings. Browser checks cover three choices, second-option selection, follow-up changes and mobile layout.

3.6 Startup and presentation improvements

Prompt excerpt: "is there a wayto start it with starting the app as well"

Response summary: The Windows launcher starts or reuses Ollama before starting SmartBrew. Model installation is a one-time setup; normal startup uses the installed weights. Manual ordering remains available when the optional model is unavailable.

Further refinement: Three example prompts fill the input for a quick demonstration. A loading indicator shows that inference is in progress and respects reduced-motion settings. The README documents installation, startup, tests and all ten feature demonstrations.

4. Critical evaluation

LLM assistance accelerated the first implementation and helped maintain traceability between requirements, features and documentation. Review and execution were essential: an early check based on diagram names missed incomplete requirement fields, the initial browser flow did not expose the two-customer login defect, and real-model requests revealed recipe-context errors that fixture-based tests alone had not found.

The development process therefore combined source review, customer feedback, deterministic tests and live-model checks. Suggestions such as additional infrastructure and password recovery were left outside the selected prototype scope. Unsupported caffeine claims were removed. These decisions kept the implementation aligned with the available data and the ten-feature demonstration.

The final application check passed 38 unit tests and 65 browser checks, including real local inference. A separate ZIP-extracted copy passed all 38 unit tests in a fresh Python 3.12.14 environment with the pinned dependencies. A browser smoke check on that clean installation verified example prompts, model loading feedback and mobile layout. Hardware and external-service behavior remain subject to the simulator scope described in the report.

5. Supporting records

- Main supplied prompt (MAIN_PROMPT.md): initial request and assignment instructions.
- Development conversation (DEVELOPMENT_TRANSCRIPT.md): 251 recorded messages, comprising 85 student messages and 166 Codex replies. Personal details are redacted.
- Transcript manifest (DEVELOPMENT_TRANSCRIPT_MANIFEST.json): source hashes and transcript integrity information.
- Repository verification records: QA_RESULTS.md and evidence/current/ contain test results, screenshots and model responses. The separate Gemini/Antigravity contribution is summarized above; its session transcript was not available for inclusion.
- Source code, environment setup and execution guide: https://github.com/kaantanidir/Software_Engi_1_LAB3

Appendix - Original main prompt

Original request: 20 October 2026. Source attachments: Lab 2 feedback, the submitted Lab2_G28.zip, and the supplied Lecture 2 and Lecture 3 PDFs.

G28Lab2 (1).pdf is the feedback we got. zip file contains our lab2 submission. according to feedback, and lab lectures 2 and 3, and if you need, lab2s guidelines: "Lab assignment 2:

This lab is about requirements and planning. For task A, we will continue to use the tool Astah. For task B, we recommend word or another textual tool with a PDF capability.

Task 2a:

Create a UML use case diagram for the system as described in the System description for all lab assignments , including the feature of your own design as you added during lab 1.

To be marked complete, the diagram must cover the functionality of the system and contain the following:

- At least 3 Actors
- At least 8 Use cases with:
- For each use case, a textual description (right click the use case and select "open use case description"). You may use this templateDownload this template for describing the use cases. Import it into Astah via Tool -> Template Settings -> UseCase Description -> Import and then Apply.
- Correct relationships between actors, between use cases, and between actors and use cases.
- A clear indication of the feature you added yourself (if any), for example by adding a note to the diagram.

Task 2b:

In this assignment, you will write user stories and create backlogs for the development.

Follow the following steps to complete this task:

1. Read Chapters 2 (on product backlogs), 3 (on sprint planning), and 6 (on sprint backlogs) of "Scrum and XP From the Trenches".

2. Based on the system description and your added feature, write down a list of at least 10 **user stories** for the SmartBrew system. Your user stories need to cover the complete functionality of the system according to the description. A typical user story has the form *"As a <role> I want <goal/desire> so that <benefit>";* you can refer to the template from the slides.

3. (starting from the user stories) Define a set of **requirements** covering the key functionalities of the system (the requirements should be expressed using the template from the slides)

4. Create an initial **product backlog** with the requirements (the initial product backlog contains at least ID, name, importance, estimate, description, how to demo) using the template provided in the slides.". we need to fix, and then do lab3:

Lab Assignment 3: Software Engineering with LLMs

The goal of this assignment is to engage with core software engineering tasks using Large Language Models (LLMs) as virtual assistants. You will observe, guide, and critically evaluate the software development lifecycle when augmented by AI.

You are required to complete **three main tasks**:

1. **Task A:** LLM-Assisted Requirements Elicitation

2. **Task B:** Code Generation & Implementation

3. **Task C:** Documentation & Repository Management

Choice of LLM

You are free to use **any LLM or AI assistant of your choice** (e.g., ChatGPT/GPT-4, Claude, Gemini, GitHub Copilot, Llama, etc.). However, you must explicitly document which model(s) and version(s) you used for each task.

Task A: Requirements Elicitation & Critical Comparison

1. **Elicitation with LLMs:**

Conduct a requirements elicitation process for the System Description used across all Labs using your chosen LLM. Generate the following three artifacts (matching the required format from Assignment 2):
- Functional and Non-Functional Requirements
- User Stories
- Product Backlog
2. **Prompting & Transcript:**

You must document the prompt strategy you used to generate these artifacts. Include the main prompts and key responses as an appendix or separate document.

3. **Comparison with Assignment 2:**

Compare the LLM-generated artifacts with the human-elicited version you submitted for Assignment 2.
- Highlight differences clearly (e.g., via side-by-side text, Git diff format with red/green highlights, or detailed annotated lists).
- Detail what the LLM improved, missing aspects it failed to identify, or unneeded/incorrect requirements it suggested.
- If you reject LLM suggestions, you **must provide a short justification** for each rejection.

(Note for students who did not submit Assignment 2: You may either complete Assignment 2 first and then perform the comparison, or perform the LLM elicitation directly while fully documenting and justifying every step of your interaction with the LLM).

Task B: Code Generation & Execution

1. **Functionality Selection:** Select a core set of **10 features/functionalities (not trivial)** from the requirements elicited in Task A.

2. **Implementation:** Realize/code these 10 functionalities with LLM assistance.

3. **Executable Prototype:** Ensure a working user experience. The system must run smoothly and demonstrate the selected 10 functionalities clearly. It is not required a complete portability of the project (Can work on only one specific OS, like macOS, linux and windows).

4. In case of complete code generation, you need to add the conversation. If you used the LLM just as an assistant to revise the code it is not required to include the conversation.

Task C: Repository & Documentation

1. **Git Repository:** Host your codebase in a Git repository (e.g., GitHub).

2. **Code Documentation & Comments:**

All source code must be documented with clear comments. At a minimum, every function/method must include:
- Description of purpose
- Expected Input parameters
- Expected Output / Return values

3. **README Document:**

The repository must include a clear `README.md` containing:
- **System Overview:** Brief description of the project.
- **Environment & Prerequisites:** Compiler/runtime versions, libraries, and external dependencies required to run the code.
- **Execution Guide:** Step-by-step instructions on how to build, run, and test the 10 functionalities (whether via command-line arguments, scripts, CLI, or GUI navigation steps).

General Information & Constraints

- You may use any programming language, framework, or IDE. However, remember that the evaluation takes place on an independent machine. Your project **must include clear environment setup instructions** or configuration files (e.g., `requirements.txt`, `package.json`, `pom.xml`, or Dockerfile) so it can be built and run without version mismatch issues.